Asymmetric Auto-Encoder Bottlenecks for Self-Supervised Latent Representation Learning

Project Case Study

Kashvi Agrawal

Problem statement • Work done • Implementation • Results and discussion

Summary

Auto-encoders learn representations without labels by compressing an input through a bottleneck and reconstructing it. Most designs mirror the encoder and decoder. Recent self-supervised work (for example masked auto-encoders) deliberately breaks this symmetry, often with a light decoder, on the argument that a weak decoder pushes the encoder to carry the semantic work. This project tests that idea in a controlled setting: I built an auto-encoder framework from scratch in NumPy in which encoder and decoder capacity are set independently, and compared four architectures across five bottleneck widths and five seeds, scoring the frozen latents with a linear probe, k-NN, and a latent effective-rank measure.

Headline finding: in this setting, a light decoder did not improve the latent space, and a symmetric deep auto-encoder was best whenever the bottleneck was narrow ($k \le 8$). A heavy encoder paired with a light decoder reached a much worse optimum at narrow bottlenecks, with latents that nearly collapsed to one dimension at $k = 2$. At $k \ge 16$, all models, including PCA and raw pixels, landed within seed noise of each other because the dataset is easy.

1. Problem Statement

Given unlabeled data $x \in \mathbb{R}^{64}$, learn an encoder $f: \mathbb{R}^{64} \to \mathbb{R}^{k}$ and decoder $g: \mathbb{R}^{k} \to \mathbb{R}^{64}$ by minimizing reconstruction error $\|x - g(f(x))\|^2$. The goal is not reconstruction itself but a latent code $z = f(x)$ that is useful for downstream tasks without any label supervision.

Research question. How does the capacity asymmetry between encoder and decoder, at a fixed bottleneck width k, affect the quality of the learned latent representation?

Hypotheses tested:

- **H1 (light decoder helps).** A weak decoder cannot do much work, so the encoder must produce a more linearly separable, informative code. Heavy encoder / light decoder should give better probe accuracy than a symmetric model.
- **H2 (heavy decoder compensates).** A strong decoder can unfold a simple (even linear) encoder's code, so a light encoder / heavy decoder model should reconstruct well but give a less structured latent.
- **H3 (noise helps).** A denoising objective should regularize the latent space and help most where the decoder is weak.

2. Work Done

- Built a NumPy neural-network library (dense layers, manual backpropagation, Adam), and verified the analytic gradients against finite differences (relative agreement to about 8 significant digits).
- Defined four auto-encoder architectures that differ only in encoder and decoder depth, with a linear bottleneck of width $k \in \{2, 4, 8, 16, 32\}$.
- Trained each configuration for 300 epochs with 5 random seeds (100 clean runs), plus a masking-noise denoising ablation (40 runs).
- Evaluated frozen latents with label-free training and label-based evaluation only: full-label linear probe, 10%-label linear probe, 5-NN, reconstruction MSE, and latent effective rank. Added PCA at every k and raw pixels as reference baselines.
- Produced figures and summary tables directly from the saved results files, and wrote this case study from those numbers.

3. Implementation

3.1 Data

scikit-learn Digits: 1,797 grayscale 8×8 digit images (64 features, 10 classes), scaled to [0, 1] and split 75/25 into 1,347 training and 450 test images, stratified by class. I had intended to use MNIST, but dataset downloads were blocked in my environment, so Digits (bundled with scikit-learn) was used. This is a much smaller and easier problem than MNIST, which shapes how the results should be read.

3.2 Architectures

All models use ReLU hidden layers, a linear bottleneck, and a sigmoid output; the exception is the Linear AE, which is linear throughout. Parameter counts below are for k = 2 (they grow slightly with k).

Model	Encoder	Decoder	Enc. params	Dec. params
Symmetric	64-128-64-k	k-64-128-64	16,706	16,768
Heavy encoder / light decoder	64-128-64-k	k-64	16,706	192
Light encoder / heavy decoder	64-k	k-64-128-64	130	16,768
Linear AE	64-k	k-64	130	192

3.3 Training

Mean-squared-error loss, Adam (learning rate 1e-3), batch size 64, 300 epochs, He initialization for ReLU layers. The same hyperparameters were used for every architecture; none were tuned on the test set. For the denoising ablation, each input pixel was independently zeroed with probability 0.3 while the target remained the clean image.

3.4 Evaluation metrics

- **Linear probe.** Standardize frozen latents, fit logistic regression on training latents, report test accuracy. Repeated with a stratified 10% subset of training labels to mimic a low-label regime.
- **5-NN accuracy.** Non-parametric check that class structure is local in latent space.
- **Reconstruction MSE.** Test-set mean squared error.
- **Effective rank.** Participation ratio of the latent covariance eigenvalues, $(\Sigma\lambda)^2 / \Sigma\lambda^2$. A value near k means all latent dimensions carry variance; a value near 1 signals dimensional collapse.

Code is organized as a small package (nn.py, models.py, evaluate.py) with a single run_experiments.py that regenerates every number in this report.

4. Results

4.1 Linear-probe accuracy

k	Symmetric	Heavy enc / light dec	Light enc / heavy dec	Linear AE	PCA
2	0.703 ± 0.100	0.539 ± 0.046	0.609 ± 0.040	0.552 ± 0.041	0.589
4	0.884 ± 0.022	0.812 ± 0.018	0.815 ± 0.030	0.794 ± 0.004	0.802
8	0.939 ± 0.010	0.916 ± 0.014	0.906 ± 0.009	0.901 ± 0.008	0.900
16	0.950 ± 0.007	0.954 ± 0.005	0.958 ± 0.006	0.961 ± 0.006	0.962
32	0.966 ± 0.006	0.967 ± 0.004	0.970 ± 0.003	0.968 ± 0.001	0.964

Test accuracy of a logistic-regression probe on frozen latents (mean ± sd over 5 seeds; PCA is deterministic). Raw 64-pixel input scores 0.969 under the same probe.

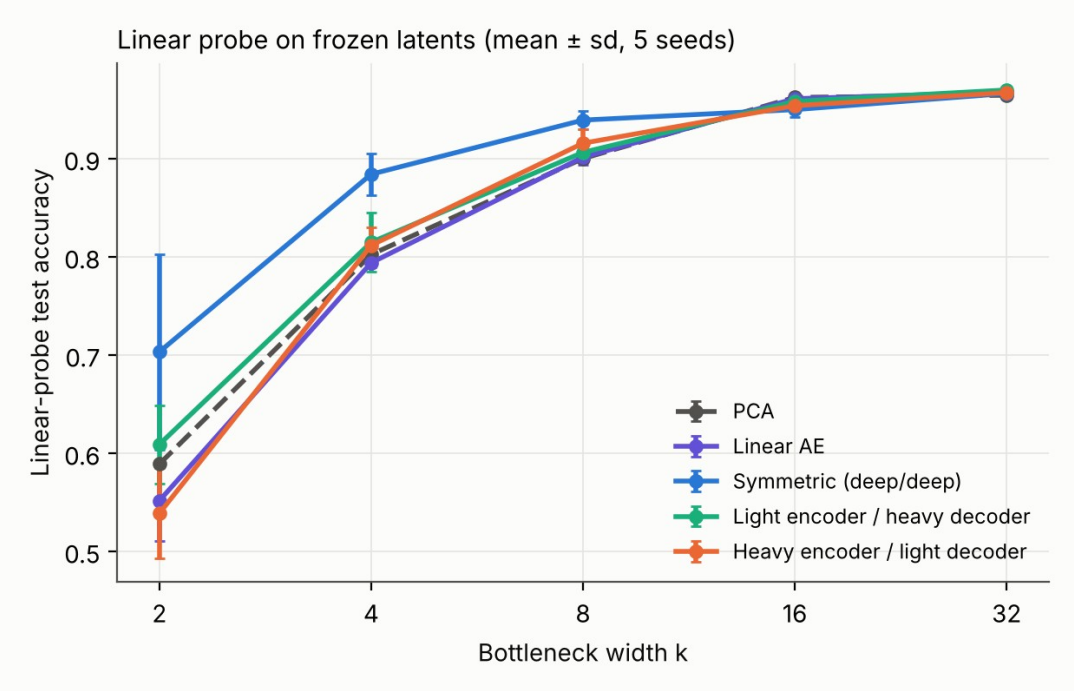


Figure 1. Linear-probe accuracy versus bottleneck width.

4.2 Low-label probe and k-NN

k	Symmetric	Heavy enc / light dec	Light enc / heavy dec	Linear AE	PCA
2	0.679	0.426	0.589	0.523	0.569
4	0.849	0.733	0.792	0.756	0.749
8	0.886	0.852	0.865	0.847	0.844
16	0.898	0.901	0.902	0.904	0.889
32	0.906	0.908	0.912	0.912	0.889

Linear-probe accuracy using 10% of training labels (raw pixels: 0.898).

k	Symmetric	Heavy enc / light dec	Light enc / heavy dec	Linear AE	PCA
2	0.832	0.564	0.639	0.560	0.616
4	0.926	0.836	0.870	0.838	0.849
8	0.962	0.927	0.953	0.941	0.936
16	0.971	0.963	0.972	0.978	0.982
32	0.975	0.977	0.978	0.975	0.969

5-NN test accuracy (raw pixels: 0.973).

4.3 Reconstruction and latent geometry

k	Symmetric	Heavy enc / light dec	Light enc / heavy dec	Linear AE	PCA
2	0.0344	0.0560	0.0428	0.0549	0.0530
4	0.0214	0.0365	0.0247	0.0401	0.0382
8	0.0125	0.0209	0.0129	0.0251	0.0246
16	0.0066	0.0081	0.0065	0.0117	0.0115
32	0.0032	0.0027	0.0026	0.0028	0.0026

Test reconstruction MSE (lower is better).

k	Symmetric	Heavy enc / light dec	Light enc / heavy dec	Linear AE	PCA
2	1.92	1.01	1.95	1.76	2.00
4	3.60	1.24	3.66	3.20	3.84
8	5.47	3.45	5.83	5.11	6.42
16	9.53	7.64	8.98	7.70	9.62

32	13.51	12.00	12.30	11.18	12.28

Effective rank of the latent codes (maximum possible equals k).

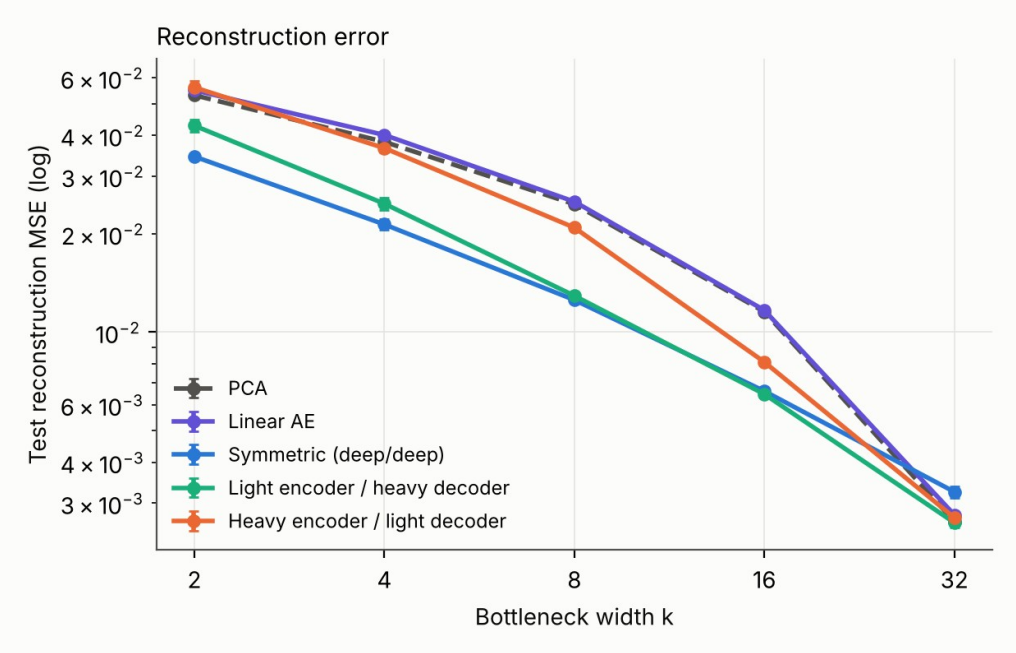


Figure 2. Reconstruction error versus bottleneck width (log scale).

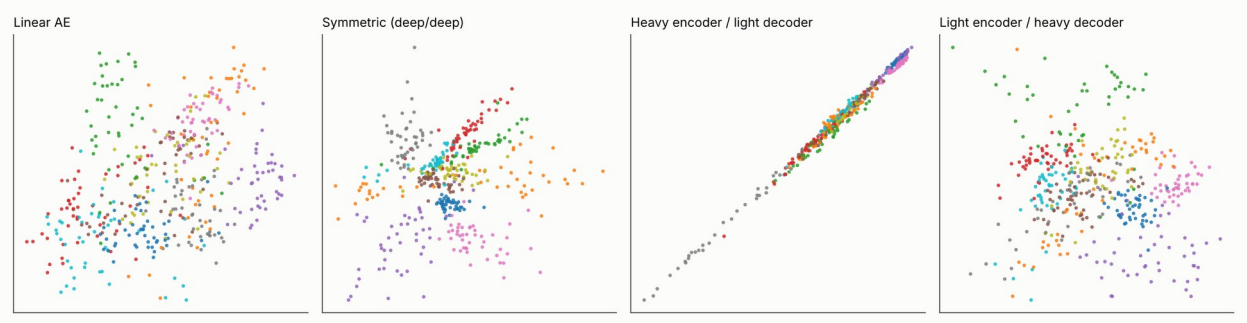


Figure 3. Two-dimensional latent codes of test digits (k = 2, seed 0), colored by digit class. Labels were never used in training.

4.4 Denoising ablation

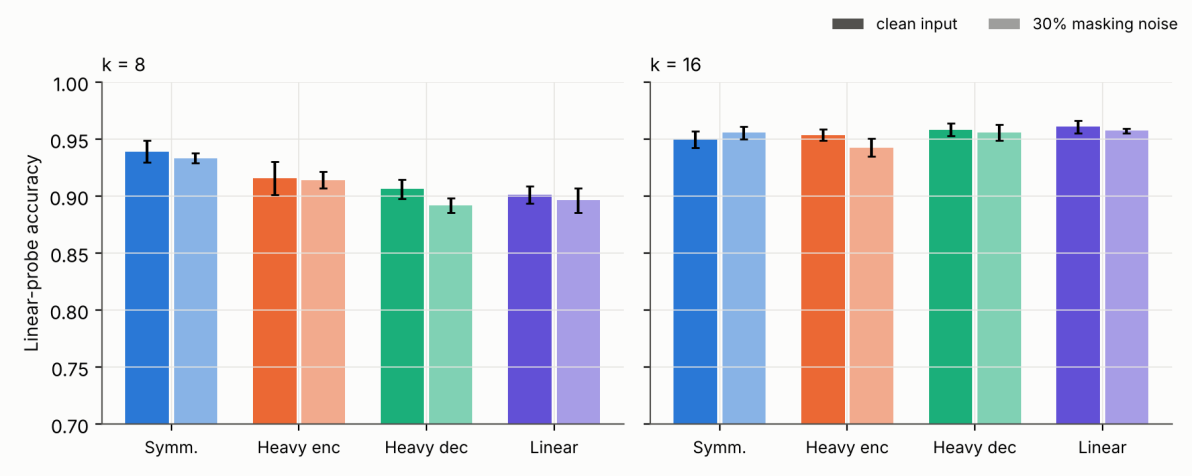


Figure 4. Linear-probe accuracy with and without 30% masking noise during training (k = 8 and 16).

5. Discussion

5.1 What the results say about the hypotheses

- **H1 (light decoder helps) is not supported.** At k = 2, 4 and 8 the heavy-encoder / light-decoder model scored 0.539, 0.812 and 0.916, against 0.703, 0.884 and 0.939 for the symmetric model. Its 10%-label probe at k = 2 (0.426) was the worst of all models. At k = 8 it was only marginally above PCA (0.916 vs 0.900) and not distinguishable from it given the seed spread.
- **H2 (heavy decoder, light encoder) is partly supported.** This model reconstructed far better than the linear models (k = 8 MSE 0.0129 vs 0.0251 for the Linear AE, almost matching the symmetric 0.0125), but its probe accuracy stayed close to PCA (0.906 vs 0.900 at k = 8). A heavy decoder can reconstruct from a code that is not especially class-structured; reconstruction quality and representation quality are different things.
- **H3 (noise helps) is not supported here.** Masking noise changed probe accuracy by at most about 0.014 in either direction. At k = 8 it lowered accuracy for all four models, by 0.002–0.014 (seed sd is about 0.005–0.014). At k = 16 the symmetric model gained 0.006 while the other three lost 0.002–0.011 (largest for heavy encoder, 0.954 to 0.943); these shifts are of the same size as seed noise. Masking also raised the clean-test reconstruction error, as expected.

5.2 Why the light decoder underperformed (and what is a hypothesis)

The effective-rank table shows the clearest pattern: with a light decoder and a deep encoder, the latents were nearly one-dimensional at k = 2 (effective rank 1.01) and strongly under-used at k = 4 (1.24 of 4). Figure 3 shows the same thing visually, with all ten digit classes lying along a single line. That is a failure to use the bottleneck rather than a clever compression. A plausible explanation, which I did not test directly, is that a single-layer decoder can only read out a linear (plus sigmoid) function of the code, so the encoder must arrange z so that this simple readout works. Early in training the strongest reconstruction gradient lies along one dominant direction, and a deep encoder can follow it and then fail

to open up additional dimensions within 300 epochs. Testing this would need longer training, a learning-rate or initialization sweep for this architecture, and a per-epoch effective-rank trace.

This differs from the masked auto-encoder setting that motivates H1. There, the decoder is light relative to a very large encoder, the input is heavily masked (about 75%), and the task is hard. Here the input is a tiny 8×8 image reconstructed in full, and the decoder is a single layer, which is probably too weak, not merely lighter. The comparison therefore says little about MAE-style design at scale.

5.3 Why the symmetric model won at narrow bottlenecks

When k is small, information is scarce and the code must be an efficient nonlinear summary of a curved data manifold. Only the symmetric model has nonlinear capacity on both sides: the encoder can bend the manifold into a compact code and the decoder can unbend it. Its advantage shrinks as k grows (0.884 → 0.939 → 0.950 at k = 4, 8, 16), and at k ≥ 16 it is no better than PCA. The light-encoder and linear models are limited by their linear encoders to PCA-like codes, which is consistent with their probe accuracies sitting at or just above PCA.

5.4 Wide bottlenecks: everything converges

At k = 16 and 32, all five representations reach 0.95–0.97 probe accuracy, which equals the raw pixel baseline (0.969). On Digits, a linear classifier on the pixels is already near ceiling, so there is nothing for a learned latent to add. Differences at these widths (under 0.012) are within seed-to-seed spread and should not be interpreted.

5.5 Limitations

- **Small and easy data.** 1,797 images at 8×8; the test set has 450 samples, so one accuracy point is about 0.2%. Results may not transfer to MNIST, CIFAR or natural images.
- **Capacity is not matched.** Asymmetric variants have very different parameter counts (about 17k vs 0.2k per side), so effects of asymmetry and total capacity are partly confounded.
- **No per-architecture tuning.** Shared hyperparameters may favor the symmetric model; the light-decoder collapse could shrink with tuning.
- **One data split.** Seeds vary initialization and shuffling but not the train/test split.
- **Only linear and k-NN evaluation,** no downstream fine-tuning, and a single noise type and level.

5.6 Next steps

- Repeat on MNIST/Fashion-MNIST or CIFAR-10 with a convolutional encoder and masked-patch objective, where the MAE-style premise actually applies.
- Add a middle-ground decoder (one hidden layer) to locate where the heavy-encoder collapse starts, and trace effective rank during training.
- Match total parameter counts, and add a regularized or variational bottleneck for comparison.

Reproducibility

All code, raw results (results.json), summary tables and figures are in the project folder asym-ae. Running run_experiments.py followed by make_figures.py regenerates every number and figure in this report in about 10 minutes on two CPU cores.